

DATA ENGINEERING - INTERVIEW PREP

PySpark Master

Deep Notes, Patterns, Performance, Streaming, Delta & Scenario
Answers

Spark 4.1.2 checked

Interviewer perspective

Production scenarios

Code patterns

60+ Q&A

How to study this document: do not memorize lines blindly. For every topic, learn the mental model, the PySpark syntax, the performance trap, and the interviewer-ready answer. In real interviews, a strong answer sounds like this: define the concept, explain why it matters in distributed processing, give a small code example, mention trade-offs, then add production checks such as data size, skew, partitions, retries, and monitoring.

Interviewer mindset: PySpark interviews are usually not testing only syntax. They test whether you can think like a data engineer: process large data safely, reduce shuffles, debug slow jobs, design incremental pipelines, handle late data, and explain your choices clearly.

Contents

1. Interviewer's Perspective
2. Spark Architecture
3. PySpark Execution Model
4. DataFrame vs RDD vs SQL
5. SparkSession & Schemas
6. DataFrame Transformations
7. Nulls, Dates & Types
8. Joins & Join Strategies
9. Aggregations & Window Functions
10. Partitions, Shuffle & Files
11. Parquet, ORC, CSV & Data Lakes
12. Performance Tuning

13. UDFs, Pandas UDFs & Arrow

14. Cache, Persist & Checkpoint

15. Structured Streaming

16. Delta Lake & Lakehouse

17. Incremental ETL & CDC

18. Production Reliability

19. Troubleshooting Scenarios

20. Pattern Cheat Sheet

21. Config Quick Reference

22. Most-Asked Interview Q&A

23. Sources Used

1. Interviewer's Perspective

A PySpark data engineering interview usually has four layers. First, the interviewer checks if you know Spark vocabulary: driver, executor, partition, stage, task, shuffle, cache, and action. Second, they test whether you can write clean DataFrame code. Third, they ask performance and production questions because real Spark jobs fail at scale. Fourth, they check data engineering judgment: incremental loads, schema evolution, idempotency, late data, orchestration, and data quality.

KEY INTERVIEW POINT

A strong PySpark answer is never only "use repartition" or "use cache." Always explain **why**: what data moves, where memory is used, what happens to partitions, and how you would verify it in the Spark UI or execution plan.

What interviewer asks	What they are really testing	Best answer style
"Why is this job slow?"	Can you find bottlenecks instead of guessing?	Check Spark UI, stage time, shuffle read/write, skew, spill, task failures, input file sizes, and query plan.
"DataFrame vs RDD?"	Do you know modern Spark best practice?	Prefer DataFrame/Spark SQL for Catalyst optimization; use RDD only for low-level custom logic.
"How do you handle duplicates?"	Can you design deterministic pipelines?	Define business key, ordering column, window row_number, keep latest, make the write idempotent.
"How do you optimize joins?"	Do you understand shuffles and data movement?	Filter early, project columns, broadcast small dimensions, handle skew, inspect physical plan.

2. Spark Architecture

Apache Spark is a distributed processing engine. In PySpark, your Python code builds a plan, Spark executes the heavy work across the cluster, and results are returned only when an action needs them.

- 1 Driver** Runs your SparkSession, builds the logical plan, schedules jobs, and coordinates executors.
- 2 Cluster Manager** Allocates resources. Examples: YARN, Kubernetes, Spark Standalone.
- 3 Executors** Worker JVM processes that run tasks, store cached data, and write shuffle output.
- 4 Partitions** Chunks of data. One task processes one partition at a time.

5 Job/Stage/Task

Action creates a job; shuffles split job into stages; stages contain many tasks.

INTERVIEW Q

Explain driver, executor, job, stage, and task.

Driver is the coordinator. Executors are worker processes. A job is created by an action like `count()` or `write()`. Spark splits a job into stages whenever a shuffle boundary appears. Each stage has tasks, and each task processes one data partition. If a task fails, Spark can retry it because transformations are tracked through lineage.

3. PySpark Execution Model

Most PySpark bugs come from forgetting that Spark is **lazy and distributed**. Transformations build a plan; actions execute the plan. A DataFrame is not the data itself - it is a distributed query plan plus schema.

Transformations: lazy

`select`, `filter`, `withColumn`, `join`, `groupBy`, `repartition`. They return a new DataFrame and do not execute immediately.

Actions: execute

`count`, `show`, `collect`, `take`, `write`. They trigger planning, scheduling, and cluster execution.

```
from pyspark.sql import functions as F

df2 = (
    spark.read.parquet("/data/orders")
    .filter(F.col("order_date") >= "2026-01-01")
    .groupBy("customer_id")
    .agg(F.sum("amount").alias("revenue"))
)

# No cluster work yet. This action triggers execution.
df2.write.mode("overwrite").parquet("/curated/customer_revenue")
```

Logical plan to physical execution

1. **Unresolved logical plan:** Spark records what you wrote.
2. **Analyzed logical plan:** Spark resolves columns, functions, table names, and types.
3. **Optimized logical plan:** Catalyst applies rules such as predicate pushdown, column pruning, constant folding, and join reordering when possible.
4. **Physical plan:** Spark chooses concrete operators such as broadcast hash join, sort merge join, hash aggregate, and file scan.
5. **Execution:** Whole-stage code generation and the execution engine run tasks across partitions.

KEY INTERVIEW POINT

Use `df.explain("formatted")` or `df.explain("cost")` to see what Spark is actually planning. Interviewers love candidates who say "I would verify with explain and Spark UI" instead of blindly tuning.

```
df2.explain("formatted")
df2.explain("cost")          # useful when table statistics are available
```

4. DataFrame vs RDD vs SQL vs Pandas

API	Use it when	Interview note
DataFrame	Most ETL, joins, aggregations, transformations, file reads/writes.	Best default for PySpark DE. It gets Catalyst optimization and a schema.
Spark SQL	SQL-heavy transformations, analysts' logic, temporary views, warehouse queries.	Same execution engine as DataFrame, so choose whichever is clearer.
RDD	Very low-level transformations, custom partition control, unsupported data structures.	RDD is powerful but misses DataFrame query optimization. Avoid unless justified.
pandas	Small local data that fits in driver memory.	Do not use <code>toPandas()</code> on large data. It collects to driver.

In Python, Spark does not expose the strongly typed Dataset API that Scala/Java have. For PySpark interviews, speak mainly in DataFrame/Spark SQL terms.

COMMON MISTAKE

Saying "RDD is always faster" is a red flag. DataFrames are usually faster and safer for structured data because Spark understands the schema and can optimize the query.

5. SparkSession & Schemas

`SparkSession` is the entry point for DataFrame and SQL work. In production ETL, always prefer explicit schemas over inference for CSV/JSON. Schema inference scans data, can infer wrong types, and can break when new files arrive with dirty values.

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, TimestampType

spark = (
    SparkSession.builder
    .appName("orders-etl")
    .getOrCreate()
)

order_schema = StructType([
    StructField("order_id", StringType(), False),
    StructField("customer_id", StringType(), False),
    StructField("order_ts", TimestampType(), True),
    StructField("amount", DecimalType(18, 2), True),
    StructField("status", StringType(), True)
])

orders = (
    spark.read
    .option("header", "true")
    .schema(order_schema)
    .csv("/landing/orders/")
)
```

INTERVIEW Q**Why should you avoid inferSchema in production?**

Because it costs extra scanning and can produce inconsistent types when files differ. A column that looks numeric in one file but contains an alphanumeric value later can fail or become null. Explicit schema makes the contract clear and makes pipeline failures easier to debug.

6. DataFrame Transformations

PySpark DataFrame code should be declarative: describe columns and transformations, let Spark optimize. Avoid Python row-by-row loops because they pull logic out of Spark's optimized engine.

```
from pyspark.sql import functions as F

clean = (
    orders
    .select(
        "order_id",
        "customer_id",
        F.to_date("order_ts").alias("order_date"),
        F.col("amount").cast("decimal(18,2)").alias("amount"),
        F.upper(F.trim("status")).alias("status")
    )
    .filter(F.col("amount") > 0)
    .withColumn(
        "status_group",
        F.when(F.col("status").isin("SHIPPED", "DELIVERED"), F.lit("COMPLETE"))
        .when(F.col("status").isin("CANCELLED", "RETURNED"), F.lit("FAILED"))
        .otherwise(F.lit("OPEN"))
    )
)
```

Need	Good PySpark pattern	Avoid
Add or replace column	<code>withColumn("x", expression)</code>	Python loop over rows
Multiple derived columns	<code>select</code> with aliases, or chained <code>withColumn</code>	Repeated actions after each column
Filter rows	<code>filter(F.col("x") > 0)</code>	Collecting and filtering in Python
Complex condition	<code>when(...).otherwise(...)</code>	Plain Python <code>if</code> on a Column

COMMON MISTAKE

A PySpark `Column` is not a Python value. You cannot write `if F.col("amount") > 0:`. Use `when`, `filter`, or SQL expressions.

7. Nulls, Dates & Types

Data engineering pipelines fail quietly when nulls, timestamps, and decimals are handled casually. Interviewers often ask these because they reveal production experience.

Null rules

- `NULL` means unknown/missing.
- Use `isNull()` and `isNotNull()`.
- Use `coalesce` for defaults.
- Use `eqNullSafe` when null equals null should match.

Date rules

- Parse strings with explicit formats.
- Be careful with time zones.
- Partition by date only when queries filter by date.
- Do not store dates as free-form strings.

```

df = (
    raw
    .withColumn("event_ts", F.to_timestamp("event_time", "yyyy-MM-dd HH:mm:ss"))
    .withColumn("event_date", F.to_date("event_ts"))
    .withColumn("amount", F.coalesce(F.col("amount"), F.lit(0)))
    .filter(F.col("event_ts").isNotNull())
)

# Null-safe join condition: true when both sides are null
joined = left.join(right, left.customer_id.eqNullSafe(right.customer_id), "left")

```

PATTERN RECOGNITION

Question says "including missing values", "unknown customer", or "null-safe match" -> mention null semantics explicitly. Use `coalesce`, `isNull`, or `eqNullSafe` depending on the requirement.

8. Joins & Join Strategies

Joins are where Spark interviews get serious because joins often cause shuffles. Your job is to reduce data before the join, choose the right join type, and know when broadcast is appropriate.

Join type	Use case	Example
inner	Keep matching rows only.	orders with valid customers
left	Keep all left rows, add right data if present.	all orders even if customer missing
left_semi	Keep left rows that have a match, but do not return right columns.	customers who placed orders
left_anti	Keep left rows that have no match.	customers with no orders
full	Keep all rows from both sides.	reconciliation between two systems

```
from pyspark.sql.functions import broadcast

# Good habit: select only needed columns before joining
orders_small = orders.select("order_id", "customer_id", "amount")
customers_small = customers.select("customer_id", "customer_tier")

# Broadcast small dimension table to avoid shuffling the large fact table
enriched = orders_small.join(
    broadcast(customers_small),
    on="customer_id",
    how="left"
)

# Anti-join: find records in landing that do not exist in target
new_records = landing.join(target.select("business_key"), "business_key", "left_anti")
```

Join strategies you should know

Strategy	How it works	When to mention
Broadcast hash join	Small table copied to every executor; large table does not shuffle.	One side is small enough to fit executor memory. Spark's default auto broadcast threshold is 10 MB unless configured.
Sort merge join	Both sides shuffle by key, sort, then merge.	Large-large equi-joins. Common default when broadcast is not possible.
Shuffle hash join	Both sides shuffle; one side per partition is hashed.	Can be efficient when one side is smaller per partition.
Broadcast nested loop	Used for some non-equi joins or cross conditions.	Potentially expensive. Explain why the condition prevents hash join.

COMMON MISTAKE

Broadcasting a table because it is "smaller" is not enough. It must fit in executor memory. If the broadcast side is too large, you can cause executor OOM or long broadcast timeouts.

PATTERN RECOGNITION

Question says "one big fact table joins a small dimension table" -> filter/project both sides, broadcast the dimension, verify with `explain` that Spark chose `BroadcastHashJoin`.

9. Aggregations & Window Functions

Aggregations usually shuffle because rows with the same key must meet on the same partition. Window functions also often shuffle and sort within partitions. These are common in DE coding rounds.

```
revenue = (
    orders
    .groupBy("customer_id")
    .agg(
        F.count("*").alias("order_count"),
        F.sum("amount").alias("total_amount"),
        F.max("order_ts").alias("last_order_ts")
    )
)

from pyspark.sql.window import Window

w = Window.partitionBy("customer_id").orderBy(F.col("order_ts").desc())

latest_order = (
    orders
    .withColumn("rn", F.row_number().over(w))
    .filter(F.col("rn") == 1)
    .drop("rn")
)
```

Question wording	Pattern	PySpark tool
"latest record per key"	rank rows inside each key	<code>row_number</code> over partition/order
"top 3 per department"	ranking with ties or no ties	<code>dense_rank</code> for ties, <code>row_number</code> for exact N rows
"running total"	ordered window frame	<code>sum().over(Window.orderBy(...).rowsBetween(...))</code>
"deduplicate by latest timestamp"	deterministic row_number	partition by business key, order by timestamp desc and tie-breaker desc

KEY INTERVIEW POINT

For deduplication, always define the business key and tie-breaker. "dropDuplicates" is fine for exact duplicates, but it is not enough when you need "latest row per customer."

10. Partitions, Shuffle & Files

A **partition** is a chunk of distributed data. A **shuffle** is data movement across executors. Shuffles are expensive because Spark writes intermediate data, transfers it over the network, and often sorts or aggregates it.

Concept	Meaning	Interview answer
Narrow transformation	Each output partition depends on one input partition.	<code>filter</code> , <code>select</code> , simple <code>map</code> . Usually no shuffle.
Wide transformation	Output depends on many input partitions.	<code>groupBy</code> , <code>join</code> , <code>distinct</code> , <code>orderBy</code> . Usually causes shuffle.
<code>repartition(n)</code>	Full shuffle to create n partitions.	Use to increase parallelism or partition by key before heavy operations/writes.
<code>coalesce(n)</code>	Usually reduces partitions without full shuffle.	Use after filtering to reduce output files, but avoid creating huge partitions.

```
# Reduce output files after a big filter
small = huge_df.filter("event_date = '2026-07-10'")
small.coalesce(20).write.mode("overwrite").parquet("/curated/events/date=2026-07-10")
```

```
# Repartition by key before writing a partitioned dataset
(
    events
    .repartition("event_date")
    .write
    .mode("append")
    .partitionBy("event_date")
    .parquet("/curated/events")
)
```

COMMON MISTAKE

Do not set partitions randomly. Too few partitions underuse the cluster and create giant tasks. Too many partitions create scheduling overhead and many small files. Tune based on data size, cluster cores, and output file target size.

11. Parquet, ORC, CSV & Data Lakes

For data engineering, file format choice matters. Columnar formats such as Parquet and ORC are preferred for analytics because Spark can read only needed columns and use metadata for

pruning.

Format	Best for	Notes
Parquet	Analytics, data lakes, Spark SQL, column pruning.	Columnar, compressed, self-describing, preserves schema.
ORC	Hive-heavy ecosystems and analytics.	Columnar and efficient.
CSV	Simple exchange/raw landing.	No strong schema, slow for analytics, quoting/escaping issues.
JSON	Semi-structured raw events.	Flexible but expensive to parse. Normalize into Parquet/Delta for analytics.

Partitioning data lake paths

Path partitioning stores files under folders like `event_date=2026-07-10/country=US/`. Spark can discover these partition columns while reading supported file sources. Good partitions match common filters and have manageable cardinality.

```
# Good: most queries filter by event_date
(
  df.write
    .mode("append")
    .partitionBy("event_date")
    .parquet("/lake/silver/events")
)
```

```
# Read only one date; Spark can prune other date folders
one_day = spark.read.parquet("/lake/silver/events").filter("event_date = '2026-07-10'")
```

COMMON MISTAKE

Do not partition by high-cardinality columns like `user_id`. You will create too many small directories/files and slow listing. Prefer date or low/medium-cardinality business columns used in filters.

12. Performance Tuning Deep Dive

Performance tuning is not magic. The priority order is: read less data, process fewer columns, reduce shuffles, handle skew, use the right join strategy, tune partitions, cache only reused data, and inspect plans/UI.

Symptom	Likely cause	What to check	Fix
One task runs much longer	Data skew	Spark UI task duration and shuffle size distribution	Salt hot keys, broadcast small side, pre-aggregate, split skewed key
Many tiny output files	Too many partitions before write	Output directory file count and average size	<code>coalesce</code> , tune write partitions, compact later
Executor OOM	Huge partition, bad broadcast, group too large, collect	Executor logs, spilled bytes, memory tab	Increase partitions, avoid collect, reduce broadcast, aggregate earlier
Long shuffle read/write	Large join/groupBy/orderBy	Stage metrics, shuffle read/write size	Filter/project early, broadcast small table, AQE, repartition by key when useful
Slow file listing	Too many small files/partitions	Driver time before stages, object store listing	Compact files, reduce partition cardinality, use table format metadata

AQE - Adaptive Query Execution

Adaptive Query Execution can re-optimize parts of the physical plan at runtime using real statistics. It can coalesce small shuffle partitions and optimize skewed joins. In modern Spark, AQE is enabled by default, but you still need to design data and joins well.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

```
# Useful while learning:  
df.explain("formatted")
```

KEY INTERVIEW POINT

A great answer to "how do you optimize Spark job?" starts with measurement: Spark UI, explain plan, stage metrics, input size, shuffle, skew, spill, and executor logs. Then tune based on evidence.

13. UDFs, Pandas UDFs & Arrow

Python UDFs are convenient but can hurt performance because Spark loses optimization visibility and data crosses the JVM/Python boundary. Built-in SQL functions are almost always better.

Approach	Performance	Use when
Built-in functions	Best	Most transformations: string, date, array, JSON, math, conditional logic.
SQL expression	Best	Readable complex expressions and functions.
Python UDF	Usually slowest	Only when no built-in equivalent exists.
Pandas UDF	Better than row UDF for vectorized work	Vectorized Python/Pandas logic using Arrow transfer.

```
# Prefer built-in functions
df = df.withColumn("email_domain", F.lower(F.split("email", "@").getItem(1)))

# Pandas UDF example - use only when built-ins cannot express the logic
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def normalize_amount(s: pd.Series) -> pd.Series:
    return (s - s.mean()) / s.std()

df.select(normalize_amount("amount").alias("amount_zscore"))
```

Even with Arrow, `toPandas()` and `toArrow()` collect all records to the driver. Use them only on small subsets.

14. Cache, Persist & Checkpoint

Caching stores a DataFrame/RDD for reuse. It is useful when the same expensive result is used multiple times. It is harmful when used blindly because cached data competes with execution memory.

```
expensive = (
    events
    .filter("event_date >= '2026-01-01'")
    .join(customers, "customer_id")
    .select("customer_id", "event_type", "event_ts")
)

expensive.cache()
expensive.count()      # materializes cache

daily = expensive.groupBy(F.to_date("event_ts").alias("dt")).count()
by_type = expensive.groupBy("event_type").count()

expensive.unpersist()
```

Tool	Purpose	Interview wording
<code>cache()</code>	Persist with default storage level.	Use only for reused DataFrames after expensive transformations.
<code>persist(level)</code>	Choose memory/disk/serialized storage level.	Use when memory is limited or recomputation is expensive.
checkpoint	Materialize and truncate lineage.	Useful for very long lineage or streaming state/checkpoints.

COMMON MISTAKE

Calling `cache()` alone does not immediately store data. It is lazy; an action such as `count()` materializes it.

15. Structured Streaming

Structured Streaming lets you write streaming logic using DataFrame-like APIs. A streaming DataFrame represents unbounded data. Spark runs the query incrementally using triggers, maintains state when needed, and uses checkpointing for recovery.

```
from pyspark.sql import functions as F

schema = "order_id string, customer_id string, event_time timestamp, amount double"

stream = (
    spark.readStream
    .schema(schema)
    .json("/landing/orders_stream")
)

agg = (
    stream
    .withWatermark("event_time", "30 minutes")
    .groupBy(F.window("event_time", "10 minutes"), "customer_id")
    .agg(F.sum("amount").alias("revenue"))
)

query = (
    agg.writeStream
    .format("parquet")
    .option("path", "/curated/order_revenue_10min")
    .option("checkpointLocation", "/checkpoints/order_revenue_10min")
    .outputMode("append")
    .start()
)
```

Streaming terms interviewers expect

Term	Meaning	Why it matters
Trigger	When Spark processes available data.	Controls latency vs cost.
Checkpoint	Stores offsets, progress, and state metadata.	Required for recovery and avoiding reprocessing chaos.
Watermark	Bound on how late data can arrive for stateful operations.	Lets Spark drop old state and control memory.
Append mode	Writes only finalized/new rows.	Common for file sinks and finalized window results.
Update mode	Writes rows changed since last trigger.	Useful for updatable sinks, not all sinks support it.
Complete mode	Writes whole result table every trigger.	Aggregation use cases, can be expensive.

KEY INTERVIEW POINT

For stateful streaming, always discuss checkpointing, watermarking, output mode, sink guarantees, and idempotency. That is how you sound production-ready.

16. Delta Lake & Lakehouse

Many data engineering roles use Spark with Delta Lake. Delta Lake adds table reliability features on top of data lakes: ACID transactions, scalable metadata handling, schema enforcement, time travel, and merge/upsert support.

```
from delta.tables import DeltaTable
from pyspark.sql import functions as F

target = DeltaTable.forPath(spark, "/lake/silver/customers")
updates = spark.read.parquet("/landing/customer_updates")

(
    target.alias("t")
    .merge(
        updates.alias("s"),
        "t.customer_id = s.customer_id"
    )
    .whenMatchedUpdate(set={
        "email": "s.email",
        "updated_at": "s.updated_at"
    })
    .whenNotMatchedInsert(values={
        "customer_id": "s.customer_id",
        "email": "s.email",
        "updated_at": "s.updated_at"
    })
    .execute()
)
```

SCD Type 1 vs Type 2

Type	Meaning	PySpark/Delta approach
SCD Type 1	Overwrite old value, no history.	MERGE with update on matching key.
SCD Type 2	Keep history by expiring old row and inserting new current row.	Use effective_from, effective_to, is_current, hash comparison, and transactional merge logic.

Delta **DELETE** or **UPDATE** removes data from the latest table version, but old files may remain physically until retention and vacuum policies allow cleanup. Know this for privacy and storage questions.

17. Incremental ETL & CDC

Most real pipelines are incremental. The core idea is to process only new or changed records, make the write idempotent, and update the watermark only after successful completion.

- 1 Read watermark** Last successful timestamp, offset, file marker, or CDC version.
- 2 Extract changes** Read rows where modified_at is greater than last watermark, or read CDC feed.
- 3 Deduplicate** Keep latest per business key using event time and tie-breaker.
- 4 Merge/write** Use idempotent upsert or overwrite only affected partitions.
- 5 Commit watermark** Update only after target write and quality checks pass.

```
from pyspark.sql.window import Window

last_watermark = "2026-07-09 00:00:00"

changes = source.filter(F.col("modified_at") > F.lit(last_watermark))

w = Window.partitionBy("business_key").orderBy(F.col("modified_at").desc(), F.col("ingest_id")
latest_changes = changes.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")

# Then MERGE latest_changes into target table
```

PATTERN RECOGNITION

Question says "daily pipeline", "only changed data", "avoid duplicates", "rerun safely" -> answer with watermark + dedup + idempotent merge + audit table + update watermark after success.

18. Production Reliability

Production PySpark is not just transformations. It includes observability, retries, data validation, schema contracts, and safe writes.

Area	What to do	Why interviewer cares
Data quality	Check nulls, duplicates, row counts, referential rules, accepted values.	Prevents silent bad data.
Idempotency	Rerun should not create duplicates or corrupt target.	Real jobs fail and rerun.
Observability	Log input/output counts, bad rows, duration, watermark, target version.	Makes incidents diagnosable.
Safe writes	Write to temp/staging path, validate, then commit or merge.	Avoids partial output.
Secrets	Use secret manager or environment injection, not hardcoded credentials.	Security baseline.

```
# Simple quality checks pattern
total = df.count()
bad_keys = df.filter(F.col("customer_id").isNull()).count()

if total == 0:
    raise ValueError("No input records found")
if bad_keys > 0:
    raise ValueError(f"Found {bad_keys} records with null customer_id")
```

19. Troubleshooting Scenarios

SCENARIO

A Spark job that normally runs in 20 minutes is now taking 2 hours. What do you do?

I would first compare input size and file count with previous runs. Then I would open Spark UI and identify the slowest stage. If one or few tasks are slow, I suspect skew. If all tasks are slow with high shuffle, I inspect join/aggregation strategy. If there is spill or OOM, I tune partitions, memory, and reduce wide data. I would also check source changes, schema changes, small files, cluster resource availability, and whether AQE/broadcast behavior changed. Finally, I would fix the root cause and add monitoring so the same issue is caught earlier.

SCENARIO

Your job creates thousands of tiny Parquet files. How do you fix it?

First find why: too many input partitions, high-cardinality partitionBy columns, or streaming micro-batches writing too often. Fix by reducing partitions before write with `coalesce` when safe, tuning shuffle partitions, choosing better partition columns, batching writes, or running compaction. The target is fewer files with healthy size, not a single giant file.

SCENARIO

One join key has 70 percent of all rows. What happens and how do you solve it?

That key causes skew: one reducer receives a huge amount of data, so one task runs very long or fails. Solutions include broadcasting the small side if possible, salting the hot key, pre-aggregating before join, splitting the hot key into a separate path, enabling AQE skew handling, or redesigning the data model.

20. Pattern Cheat Sheet

Interview problem	Use this pattern	Key PySpark tools
Latest row per entity	Window rank then filter	<code>Window.partitionBy().orderBy()</code> , <code>row_number</code>
Deduplicate exact rows	Drop duplicate rows/keys	<code>dropDuplicates</code>
Deduplicate keeping latest	Business key + ordering tie-breaker	<code>row_number</code> , deterministic order
Customers with no orders	Anti-join	<code>join(..., "left_anti")</code>
Fact + small dimension	Broadcast join	<code>broadcast(dim)</code> , <code>explain</code>
Slow large aggregation	Reduce columns, tune shuffle, handle skew	<code>groupBy</code> , <code>spark.sql.shuffle.partitions</code> , AQE
Incremental load	Watermark + merge	high-watermark table, Delta <code>merge</code>
Late streaming data	Event-time watermark	<code>withWatermark</code> , output mode, checkpoint
Nested JSON flattening	Select nested fields, explode arrays	<code>col("a.b")</code> , <code>explode</code> , <code>from_json</code>
Small file problem	Coalesce/compact/repartition smarter	<code>coalesce</code> , optimized write, compaction job

21. Config Quick Reference

Config	Default in current docs	Use
<code>spark.sql.shuffle.partitions</code>	200	Number of partitions for shuffle joins/aggregations. Tune based on data and cluster.
<code>spark.sql.files.maxPartitionBytes</code>	128 MB	Max bytes per input partition for file sources such as Parquet/JSON/ORC.
<code>spark.sql.autoBroadcastJoinThreshold</code>	10 MB	Max size Spark may auto broadcast for joins. Set -1 to disable.
<code>spark.sql.adaptive.enabled</code>	true	Enable AQE runtime re-optimization.
<code>spark.sql.adaptive.advisoryPartitionSizeInBytes</code>	64 MB	Target size for AQE shuffle partition decisions.
<code>spark.sql.execution.arrow.pyspark.enabled</code>	false	Enable Arrow optimization for pandas conversion paths.

Do not recite configs as magic numbers. Say: "I tune this after measuring input size, shuffle size, task duration, cluster cores, and spill."

22. Most-Asked Interview Q&A

Fundamentals

INTERVIEW Q 1**What is PySpark?**

PySpark is the Python API for Apache Spark. It lets Python developers build distributed batch, SQL, streaming, and ML workloads on Spark. In data engineering, we mostly use PySpark DataFrames and Spark SQL for ETL over large datasets.

INTERVIEW Q 2**Why is Spark faster than traditional MapReduce?**

Spark can keep intermediate data in memory, builds DAGs instead of forcing every step to write to disk, and has an optimized SQL/DataFrame engine. MapReduce is more rigid: map, shuffle, reduce, write. Spark still writes during shuffles, but it avoids unnecessary disk writes between every step.

INTERVIEW Q 3**What is lazy evaluation?**

Spark does not execute transformations immediately. It records them in a plan. Execution starts when an action runs. This allows Spark to optimize the whole plan, combine operations, push filters, prune columns, and avoid unnecessary work.

INTERVIEW Q 4**What is lineage?**

Lineage is Spark's record of how a DataFrame/RDD was derived from sources. It enables fault tolerance: if a partition is lost, Spark can recompute it from previous transformations instead of starting the whole application manually.

INTERVIEW Q 5**What is the difference between transformation and action?**

Transformations return a new DataFrame and are lazy, for example select, filter, join, groupBy. Actions trigger execution and return results or write output, for example count, show, collect, take, write.

INTERVIEW Q 6**What is a partition?**

A partition is a slice of distributed data. Spark runs one task per partition per stage. Partition sizing controls parallelism, memory pressure, and output file count.

INTERVIEW Q 7**What is a shuffle?**

A shuffle is redistribution of data across partitions/executors, usually by key. Joins, groupBy, distinct, and orderBy often cause shuffles. Shuffles are expensive because they involve disk, network, serialization, and often sorting.

INTERVIEW Q 8**What is the difference between job, stage, and task?**

An action creates a job. Spark splits a job into stages at shuffle boundaries. A stage contains tasks, where each task processes one partition.

INTERVIEW Q 9**What is SparkSession?**

SparkSession is the main entry point for DataFrame and SQL APIs. It wraps SparkContext and provides access to reading data, creating DataFrames, running SQL, and managing catalog/session configuration.

INTERVIEW Q 10**What is Catalyst optimizer?**

Catalyst is Spark SQL's optimizer. It analyzes and optimizes logical plans using rules and statistics, then helps choose physical plans. It is a major reason DataFrames/Spark SQL are preferred over low-level RDD code for structured data.

DataFrame Coding

INTERVIEW Q 11**How do you select and rename columns?**

Use `select` with `alias`: `df.select(F.col('customer_id').alias('id'))`. This is better than selecting all columns when you only need a few because it enables column pruning.

INTERVIEW Q 12**How do you add a conditional column?**

Use `when` and `otherwise`, not Python `if`. Example: `F.when(F.col('amount') > 1000, 'HIGH').otherwise('NORMAL')`.

INTERVIEW Q 13**How do you handle null values?**

Use `isNull`, `isNotNull`, `coalesce`, `fillna`, or null-safe equality depending on the case. Never compare with Python `None` using normal equality for filter logic.

INTERVIEW Q 14**How do you read CSV safely?**

Specify schema, header, delimiter, quote/escape options, bad-record handling, and encoding if needed. Avoid relying on `inferSchema` in production.

INTERVIEW Q 15**How do you flatten nested JSON?**

Read JSON with a schema, select nested fields using dot notation, use `explode` or `explode_outer` for arrays, and normalize into a stable curated schema.

INTERVIEW Q 16**How do you deduplicate exact duplicates?**

Use `dropDuplicates()` for entire row duplicates or `dropDuplicates(['key1', 'key2'])` for duplicate keys when any row can be kept.

INTERVIEW Q 17**How do you keep latest record per key?**

Use a window: partition by business key, order by update timestamp descending and a deterministic tie-breaker, assign `row_number`, keep rn=1.

INTERVIEW Q 18**When should you use Spark SQL instead of DataFrame API?**

Use SQL when the logic is clearer, already owned by analysts, or involves SQL-heavy transformations. DataFrame and SQL use the same underlying engine, so performance is usually determined by the plan, not syntax style.

INTERVIEW Q 19**What does collect() do and why is it dangerous?**

`collect()` brings all rows to the driver. It can crash the driver for large data. Use `show`, `take`, `limit`, or write distributed output instead.

INTERVIEW Q 20**How do you compare two DataFrames?**

For data reconciliation, compare counts, schemas, keys, aggregates, and use anti-joins in both directions. For exact equality, align schemas/order if needed and use except/subtract patterns, but remember row ordering is not guaranteed unless explicitly sorted.

Joins and Aggregations

INTERVIEW Q 21**How do you optimize a join?**

Project only needed columns, filter early, broadcast small dimensions, ensure join keys have matching types, inspect the physical plan, handle skew, and avoid unnecessary many-to-many joins.

INTERVIEW Q 22**What is broadcast join?**

Spark sends the small table to every executor, allowing each partition of the large table to join locally. It avoids shuffling the large table but requires the broadcast side to fit in memory.

INTERVIEW Q 23**What is left semi join?**

It returns rows from the left DataFrame that have a match on the right, without returning right columns. It is useful for existence checks and can be more efficient than inner join followed by selecting left columns.

INTERVIEW Q 24**What is left anti join?**

It returns rows from the left DataFrame that do not have a match on the right. It is the PySpark pattern for 'records missing in target' or 'customers with no orders'.

INTERVIEW Q 25**How do you handle data skew in joins?**

Detect skew in Spark UI by task duration and shuffle size. Fix with broadcast if possible, salting hot keys, pre-aggregation, splitting hot keys, AQE skew handling, or changing the data model.

INTERVIEW Q 26**Why can groupBy be expensive?**

Rows with the same key must be brought together, so groupBy usually causes shuffle. High-cardinality keys, skewed keys, and wide rows make it expensive.

INTERVIEW Q 27**row_number vs rank vs dense_rank?**

`row_number` gives unique sequence even for ties. `rank` leaves gaps after ties. `dense_rank` does not leave gaps. Use `dense_rank` when top-N should include ties.

INTERVIEW Q 28**How do you calculate running total?**

Use a window ordered by time with frame from unbounded preceding to current row, then apply `sum` over that window.

INTERVIEW Q 29**What is a Cartesian join and why is it dangerous?**

A Cartesian join combines every row from one side with every row from the other. It can explode row counts massively. Use only when intentional and bounded.

INTERVIEW Q 30**How do you avoid duplicate rows after a join?**

Check join key uniqueness. If the right side has multiple rows per key, the join multiplies rows. Deduplicate or aggregate the right side before joining, or include more keys in the join condition.

Performance

INTERVIEW Q 31**How do you optimize a slow Spark job?**

Measure first: Spark UI, slow stages, task skew, shuffle, spill, input size, file count, and explain plan. Then reduce input/columns, filter early, optimize joins, tune partitions, handle skew, cache reused data, and fix small files.

INTERVIEW Q 32**What is AQE?**

Adaptive Query Execution re-optimizes query plans at runtime using actual statistics. It can coalesce shuffle partitions, change join strategies, and optimize skewed joins. It helps but does not replace good data design.

INTERVIEW Q 33**When should you cache?**

Cache when the same expensive DataFrame is reused multiple times. Do not cache one-time data. Materialize the cache with an action and unpersist after use.

INTERVIEW Q 34**cache vs persist?**

`cache()` is shorthand for default persistence. `persist()` lets you choose storage level such as memory and disk. Use persist when memory pressure or recomputation cost requires a different storage strategy.

INTERVIEW Q 35**repartition vs coalesce?**

`repartition` does a full shuffle and can increase or decrease partitions. `coalesce` usually avoids full shuffle and is used to reduce partitions. Use repartition for balanced redistribution, coalesce for reducing output files after filtering.

INTERVIEW Q 36**Why is SELECT * bad in Spark?**

It reads and carries unnecessary columns through the plan. In columnar formats, selecting only needed columns enables column pruning and reduces IO, memory, network, and shuffle size.

INTERVIEW Q 37**What is predicate pushdown?**

Spark pushes filters down to the data source when possible so less data is read. With Parquet, metadata can help skip row groups/files depending on the predicate and statistics.

INTERVIEW Q 38**What is column pruning?**

Spark reads and processes only columns required by the query. This is very effective with columnar formats like Parquet and ORC.

INTERVIEW Q 39**What causes executor OOM?**

Huge partitions, skew, bad broadcast, collecting too much data, large group state, excessive caching, or too few partitions. Fix by reducing data, increasing partitions, avoiding collect, tuning memory, and handling skew.

INTERVIEW Q 40**How do you reduce small files?**

Avoid over-partitioned writes, use coalesce before writing when safe, choose partition columns carefully, compact files, and avoid tiny streaming micro-batch files when possible.

Storage and Pipeline Design

INTERVIEW Q 41**Why is Parquet preferred for analytics?**

It is columnar, compressed, self-describing, and supports efficient reads with column pruning and metadata. It is much better for repeated analytics than CSV/JSON.

INTERVIEW Q 42**What is partition pruning?**

When a dataset is partitioned by a folder/table column and the query filters on that column, Spark can skip non-matching partitions and read far less data.

INTERVIEW Q 43**How do you choose partition columns?**

Choose columns frequently used in filters, usually date or region-like columns. Avoid high-cardinality columns that create too many small files/directories.

INTERVIEW Q 44**What is schema evolution?**

Schema evolution means the table schema changes over time, for example adding columns. Handle it deliberately with compatibility rules, schema validation, and controlled merge/evolution settings.

INTERVIEW Q 45**Bronze, Silver, Gold layers?**

Bronze stores raw or near-raw ingested data. Silver cleans, deduplicates, conforms, and enriches data. Gold contains business-ready aggregates or dimensional models for analytics/BI.

INTERVIEW Q 46**How do you build an idempotent pipeline?**

Use deterministic keys, deduplicate input, use merge/upsert or overwrite affected partitions, avoid append-only duplicates, record audit metadata, and update watermarks only after successful commit.

INTERVIEW Q 47**How do you implement incremental load?**

Read last successful watermark, extract changed rows, deduplicate by business key, merge into target, validate counts, then commit the new watermark.

INTERVIEW Q 48**How do you handle late-arriving data?**

In batch, reprocess an overlap window and merge idempotently. In streaming, use event-time watermarking and choose acceptable lateness based on business rules.

INTERVIEW Q 49**What is CDC?**

Change Data Capture records inserts, updates, and deletes from a source system. In Spark pipelines, CDC is often applied using merge/upsert logic into a target table, preserving idempotency and order.

INTERVIEW Q 50**What is SCD Type 2?**

Slowly Changing Dimension Type 2 keeps history by creating a new version when attributes change. Common columns are `effective_from`, `effective_to`, `is_current`, and hash/checksum for change detection.

Streaming and Delta

INTERVIEW Q 51**What is Structured Streaming?**

Spark's DataFrame-based streaming engine. You define transformations like batch DataFrames, and Spark incrementally processes new data using triggers, checkpoints, and state management.

INTERVIEW Q 52**What is a watermark?**

A watermark tells Spark how long to wait for late event-time data. It bounds state size by allowing Spark to drop old state once data is older than the watermark threshold.

INTERVIEW Q 53**Why is `checkpointLocation` important?**

It stores progress, offsets, and state needed to recover after failure. Without a stable checkpoint, a restarted streaming query may reprocess or lose track of progress.

INTERVIEW Q 54**Append vs Update vs Complete output mode?**

Append outputs only final/new rows. Update outputs rows changed since last trigger. Complete outputs the whole result table each trigger. Available modes depend on query type and sink.

INTERVIEW Q 55**What is Delta Lake?**

Delta Lake is an open-source storage layer for data lakes that adds ACID transactions, metadata handling, schema enforcement/evolution, time travel, and merge/update/delete support.

INTERVIEW Q 56**Why use Delta merge?**

Merge supports upserts: update matching target records and insert new records in one atomic operation. It is central for CDC, deduped incremental loads, and SCD implementations.

INTERVIEW Q 57**What is time travel in Delta?**

Time travel lets you read an older table version or timestamp. It is useful for rollback, audit, reproducibility, and debugging bad data loads.

INTERVIEW Q 58**What does vacuum do?**

Vacuum removes old physical files no longer needed by the latest table versions after retention rules. It can reduce storage but may limit time travel if old files are removed.

Production Scenarios

INTERVIEW Q 59**Your pipeline failed halfway. How do you rerun safely?**

Design for idempotency. Read the same input range, deduplicate, write with merge or overwrite affected partitions, and update audit/watermark only after success. Avoid plain append without keys.

INTERVIEW Q 60**How do you monitor a PySpark job?**

Capture input/output counts, rejected rows, job duration, Spark app ID, target version/path, watermark, schema version, and error logs. Use Spark UI/history server for stage/task metrics and executor health.

INTERVIEW Q 61**How do you handle bad records?**

Separate invalid records into a quarantine path/table with reason codes, keep counts, fail the job if thresholds are exceeded, and make business rules explicit.

INTERVIEW Q 62**What is your approach to data quality?**

Define checks for schema, nullability, uniqueness, referential integrity, accepted values, volume anomalies, and freshness. Run checks before publishing curated data and store metrics for observability.

INTERVIEW Q 63**How do you explain PySpark project experience in interview?**

State the business problem, data volume, sources/sinks, transformations, performance challenges, reliability design, orchestration, and measurable outcome. Mention trade-offs and what you monitored.

INTERVIEW Q 64**What are red flags in PySpark code review?**

Unbounded collect/toPandas, Python UDF where built-ins exist, no explicit schema, blind repartition/cache, writing too many small files, non-idempotent appends, missing checkpoint for streaming, and no data quality checks.

Final interview formula: when asked a scenario, answer in this order: clarify data size and requirement, identify the distributed operation, write the clean DataFrame pattern, explain performance implications, and finish with production checks. This structure makes you sound calm and senior.

23. Sources Used

These notes were synthesized from current official documentation and written in interview-prep language. Checked on July 10, 2026.

- [Apache Spark PySpark Overview - Spark 4.1.2](#)
- [Apache Spark SQL, DataFrames and Datasets Guide](#)
- [Apache Spark SQL Performance Tuning](#)
- [Apache Spark Parquet Files Guide](#)
- [Apache Spark Structured Streaming Programming Guide](#)
- [Apache Arrow in PySpark](#)
- [Delta Lake Documentation](#)

PySpark Master - Data Engineering Interview Notes

Printable HTML document. Open in Chrome and use Ctrl+P to save as PDF.